

neonity-agent

A lightweight AI coding agent framework with multi-provider LLM support, inspired by pi-mono's architecture.

Features

-  **Multi-turn Conversation Loop** - REACT pattern: Reason → Act → Observe
-  **Tool Calling** - Parallel execution of multiple tools
-  **Multi-provider Support** - OpenAI, Anthropic, Gemini, GLM, MiniMax
-  **Streaming Responses** - Real-time display of AI thinking and execution
-  **Event-driven** - Complete event system for monitoring and debugging
-  **Context Protection** - 80% threshold automatic trimming to prevent overflow
-  **Built-in Tools** - terminal, read_file, write_file, git_*, ls

Installation

```
npm install
npm run build
```

Configuration

Create a `.env` file based on `.env.example` :

```
OPENAI_API_KEY=your_openai_api_key
ANTHROPIC_API_KEY=your_anthropic_api_key
GEMINI_API_KEY=your_gemini_api_key
GLM_API_KEY=your_glm_api_key
MINIMAX_API_KEY=your_minimax_api_key
```

Usage

```
# Development mode
npm run dev

# Build and run
npm run build
npm start

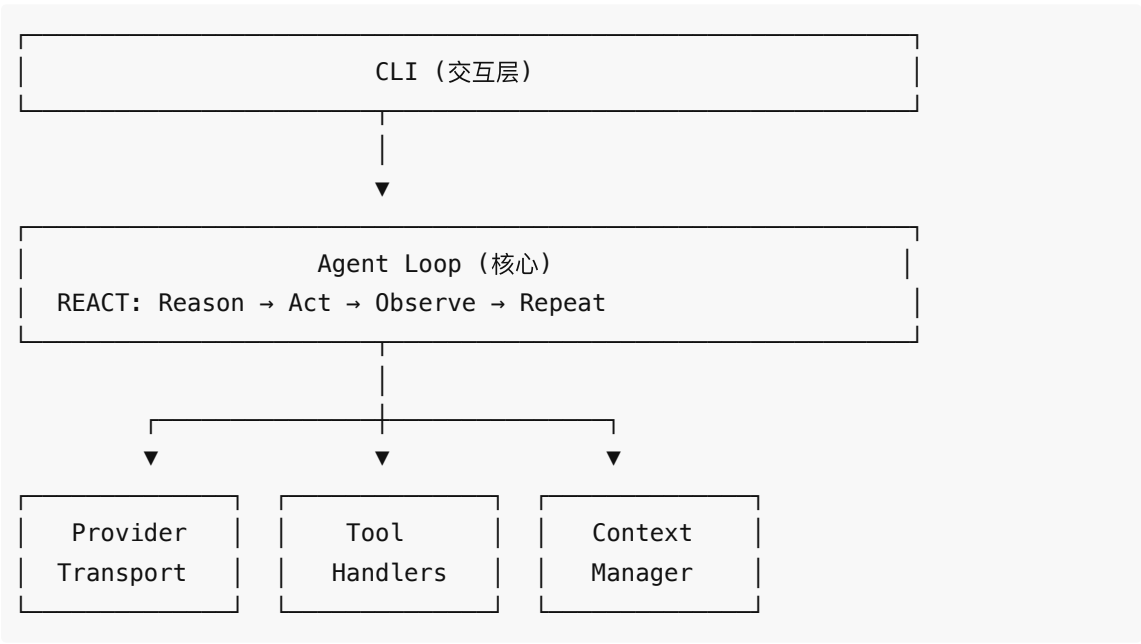
# Type checking
npm run typecheck
```

Built-in Tools

Tool	Description
terminal	Execute shell commands
read_file	Read file contents
write_file	Write content to files
git_status	Show Git repository status
git_log	Show commit history
git_diff	Show file differences
git	Execute arbitrary Git commands
ls	List directory contents

See [TOOLS_REFERENCE.md](#) for detailed documentation.

Architecture



Context Management

The agent includes automatic context window protection:

- **Token Estimation:** Optimized for mixed Chinese/English content
- **80% Threshold:** Automatically trims when approaching context limit
- **FIFO Strategy:** Preserves recent messages by default

Cost Router

Intelligent model routing inspired by [FrugalGPT](#):

```
User query → classify complexity (cheap model) → route to cheap/expensive model
```

- **Default:** MiniMax M2.7 (cheap) → GLM-5.1 (expensive), threshold=4
- **Strategies:** pre-classify (default), always-cheap , always-expensive

```
# Enable router
neonity --router

# Slash commands
/router stats          # Show routing statistics
/router test <query>   # Classify a query without running
/router strategy <s>   # Switch strategy
/router reset          # Reset statistics
```

Documentation

- [LEARNING GUIDE.md](#) - Complete learning path
- [TOOLS REFERENCE.md](#) - Tool reference documentation

Project Structure

```
neonity-agent/
├─ src/
│   ├─ agent/          # Agent loop implementation
│   ├─ cli/            # Command-line interface
│   ├─ context/        # Context management
│   ├─ provider/       # LLM provider transports
│   ├─ router/         # Cost-aware model routing
│   └─ tool/           # Built-in tools
├─ .agent-output/     # Agent-generated artifacts
│   └─ artifacts/docs/ # Documentation
└─ package.json
```

Quick Start

1. Basic Conversation

```
import { AgentLoop } from './src/index.js';
import { openai } from './src/provider/registry.js';

const agent = new AgentLoop({
  model: openai("gpt-4o"),
```

```
    systemPrompt: "You are a helpful AI assistant.",
    tools: [],
    maxIterations: 10,
  });

const result = await agent.run("Hello, please introduce yourself");
console.log(result.messages);
```

2. Using Tools

```
import { terminalTool, readFileTool, writeFileTool } from
"./src/tool/index.js";

const agent = new AgentLoop({
  model: openai("gpt-4o"),
  systemPrompt: "You are a coding assistant that can use tools to operate
files and execute commands.",
  tools: [terminalTool, readFileTool, writeFileTool],
  maxIterations: 10,
});

await agent.run("Please help me check the file list in the current
directory");
```

Core Concepts

Message Types

```
// User message
interface UserMessage {
  role: "user";
  content: string;
}

// Assistant message
interface AssistantMessage {
  role: "assistant";
  content: string;
  toolCalls?: ToolCall[];
  stopReason?: StopReason;
  usage?: Usage;
}

// Tool execution result
interface ToolResultMessage {
```

```
    role: "tool_result";
    toolCallId: string;
    content: string;
    isError?: boolean;
  }
```

Tool Calling

Tool definition and handler:

```
interface ToolDefinition {
  name: string;
  description: string;
  parameters: Record<string, unknown>; // JSON Schema
}

interface ToolHandler {
  definition: ToolDefinition;
  execute(args: Record<string, unknown>): Promise<string>;
}
```

Streaming Events

```
type StreamEvent =
  | { type: "start"; message: AssistantMessage }
  | { type: "text_delta"; text: string }
  | { type: "tool_call_start"; index: number; id: string; name: string }
  | { type: "tool_call_delta"; index: number; arguments: string }
  | { type: "usage"; usage: Usage }
  | { type: "done"; message: AssistantMessage }
  | { type: "error"; error: Error; message: AssistantMessage };
```

Learning Path

Recommended order for learning this project:

Phase 1: Understanding Basics (1-2 days)

- `src/types.ts` - Core type definitions
- `src/event-stream.ts` - Event stream mechanism
- `src/provider/helpers.ts` - Message transformation

Phase 2: Deep Dive into Core (2-3 days)

- `src/provider/registry.ts` - Provider registration mechanism
- `src/provider/openai.ts` - Complete implementation example
- `src/agent/loop.ts` - Core loop logic

Phase 3: Tool Extension (1-2 days)

- `src/tool/terminal.ts` - Command execution tool
- `src/tool/read-file.ts` - File reading tool
- `src/tool/write-file.ts` - File writing tool

Phase 4: Context Management (1 day)

- `src/context/manager.ts` - Context manager

For detailed learning guide, see [LEARNING GUIDE.md](#).

Design Patterns

1. Dependency Injection

```
class AgentLoop {
  constructor(config: AgentConfig) {
    this.config = config;
  }
}
```

2. Event-driven

```
await agent.run(message, (event) => {
  if (event.type === "message_delta") {
    process.stdout.write(event.delta);
  }
});
```

3. Streaming

```
async *stream(options: StreamOptions): AsyncGenerator<StreamEvent> {
  for await (const chunk of apiStream) {
    yield transform(chunk);
  }
}
```

FAQ

Q: Why are tool calls parallel?

A: Parallel execution improves efficiency, especially when tools have no dependencies.

Q: How to limit tool execution time?

A: Use the `timeout` parameter in tool implementations.

Q: How to add conversation history limits?

A: Use Context Manager to check and trim message history before each LLM call.

Q: How accurate is token estimation?

A: Current heuristic algorithm (Chinese 0.5 token/char, English 0.25 token/char) has ~80-90% accuracy.

Further Reading

- [REACT Paper](#) - Reasoning and Acting in Language Models
- [OpenAI Function Calling](#)
- [Anthropic Tool Use](#)

Contributing

Issues and Pull Requests are welcome!

License

MIT

[中文版 README](#)

Official Website (Next.js 16)

A modern, minimal official website is available in [website/](#) .

```
cd website
npm install
npm run dev
```